TW-Mailer – Basic Version

1. Client- und Server Architektur

- ClientSocket: Kapselt die Low-Level-Netzwerkschnittstelle (POSIX Sockets). Übernimmt den Verbindungsaufbau zum Server via IP-Adresse und Port, das Senden von Befehlen und Datenpaketen sowie das zeilenweise Einlesen von Serverantworten (readline).
- ProtocolSerializerValidator: Dient als Validierungs- und Serialisierungsschicht. Prüft Benutzereingaben gemäß Spezifikation (z. B. Maximallänge von 8 Zeichen für Benutzernamen und max. 80 Zeichen für den Betreff) und baut die Protokoll-Payloads regelkonform auf.
- ClientUI: Steuert das interaktive Kommandozeilen-Menü (SEND, LIST, READ, DEL, QUIT), nimmt Benutzereingaben entgegen und koordiniert den Datenfluss zwischen Benutzeroberfläche, Validierung und Socket.
- Server: Erzeugt den Server-Socket, bindet diesen an den angegebenen Ziel-Port (bind) und wartet auf eingehende Client-Verbindungen (listen). Für jede akzeptierte Verbindung (accept) wird künftig ein eigener Thread gespawnt, um Anfragen unabhängig voneinander zu bearbeiten.
- MailSpooler: Verantwortlich für die Datenhaltung (Persistierung). Verwaltet das Dateisystem innerhalb des übergebenen Spool-Verzeichnisses (mail-spool-directory), speichert eingehende Nachrichten ab, liest Benutzer-Postfächer aus und löscht spezifische E-Mails.

2. Verwendete Technologien

- Programmiersprache: C++17 (Nutzung von std::filesystem, std::string_view und RAII für Ressourcenmanagement).
- Netzwerkschnittstelle: POSIX Stream Sockets (<sys/socket.h>, <netinet/in.h>, <arpa/inet.h>) unter Linux für verbindungsorientierte TCP/IP-Kommunikation.
- Build-System: GNU Make zur automatisierten Kompilierung über ein zentrales Makefile.
- Parallelisierung: Multi-Threading via C++ Standard Library (<thread>) und POSIX Threads (-pthread), wodurch der Server mehrere Client-Verbindungen simultan verarbeiten kann.

3. Entwicklungsstrategie und benötigte Anpassungen

- Entwicklung nach dem Prinzip der Separation of Concerns (SoC):
- Dateisystem-Persistierung: Entwurf des MailSpooler. Pro Empfänger wird im Spool-Ordner ein eigenes Unterverzeichnis (z. B. mail_spool/<username>/) angelegt. Jede E-Mail wird als separate Textdatei (z. B. 1.txt, 2.txt) abgespeichert. Dies erleichtert das Abrufen (READ), Auflisten (LIST) und Entfernen (DEL) von Nachrichten ohne aufwändiges Parsing einer monolithischen Datei.
- Fehlerbehandlung: Kapselung aller Protokollregeln im ProtocolSerializerValidator, um ungültige Eingaben bereits vor dem Senden abzufangen. Der Server antwortet bei Fehlern stets deterministisch mit ERR\n und bei Erfolg mit OK\n.

- Erweiterung von Iterativ auf Concurrent (Multi-Threading): Um Blockaden bei langen Lese-/Schreibvorgängen zu vermeiden, wird die Architektur auf ein Multi-Threaded-Modell erweitert. Nach dem accept() übergibt der Haupt-Thread den Client-Socket an einen neuen Thread (std::thread).
- Threadsicherheit im MailSpooler: Durch den parallelen Zugriff mehrerer Clients auf dieselben Benutzerordner wurden Mutex-Sperren (std::mutex) im MailSpooler integriert, um Race Conditions beim gleichzeitigen Schreiben oder Löschen von Dateien zu verhindern.